

هیچ اشتباهی بخشیده نمی‌شود

- سطح: آسان

- طراح: حدیث صابری

شما باید برنامه‌ای بنویسید که نمرات دانشجویان را از ورودی خوانده و پردازش کند. در این فرآیند، انواع خطاها ممکن است رخ دهد که باید به درستی مدیریت شوند.

می‌توانید کلاس‌های اولیه پروژه را از [این لینک](#) دانلود کنید.

جزئیات پروژه

کلاس‌های استثنای سفارشی

۱. کلاس **NegativeScoreException**

◦ وقتی نمره وارد شده منفی باشد، این استثنا پرتاب می‌شود.

◦ پیام آن: "Score cannot be negative: " + score

۲. کلاس **InvalidScoreException**

◦ وقتی نمره خارج از بازه ۰ تا ۲۰ باشد، این استثنا پرتاب می‌شود.

◦ پیام آن: "Score must be between 0 and 20: " + score

۳. کلاس **EmptyStudentListException**

◦ وقتی لیست دانشجویان خالی باشد، این استثنا پرتاب می‌شود.

◦ پیام آن: "Student list is empty"

۴. کلاس **DuplicateStudentException**

◦ وقتی دانشجویی با نام تکراری وارد شود، این استثنا پرتاب می‌شود.

◦ پیام آن: "Student with name '" + name + "' already exists"

۵. کلاس **InvalidNameException**

◦ وقتی نام دانشجو خالی یا فقط شامل فاصله باشد، این استثنا پرتاب می‌شود.

◦ پیام آن: "Student name cannot be empty"

۶. کلاس **ArrayFullException**

◦ پیام آن: "Array is full. Cannot add more students"

۷. کلاس **StudentNotFoundException**

◦ پیام آن: "Student with name '" + name + "' not found"

کلاس Student

ویژگی‌ها

• **name**: از نوع String

• **score**: از نوع double

متدها

- متد سازنده که پارامترهای نام و نمره را می‌گیرد و اعتبارسنجی زیر را انجام می‌دهد:
 - اگر نام null یا خالی یا فقط شامل space بود، **InvalidNameException** پرتاب کند.
 - اگر نمره منفی بود، **NegativeScoreException** پرتاب کند.
 - اگر نمره $20 <$ بود، **InvalidScoreException** پرتاب کند.
- متدهای getter برای نام و نمره.
- متد **equals** را override کنید تا مساوی بودن دو دانشجو بر اساس نام بررسی کند.

کلاس ScoreProcessor

متدها

۱. **calculateAverage(Student[] students, int count)**
 - میانگین نمرات را محاسبه کرده و به صورت double برمی‌گرداند.
 - اگر count برابر صفر باشد، **EmptyStudentListException** پرتاب کند.
 - اگر هر جای آرایه null وجود داشت، **NullPointerException** را بگیرید و پیام "Invalid data found in array" چاپ کنید. دقت داشته باشید که فقط تا اندیس count از آرایه را پیمایش کنید
 - اگر null در آرایه وجود داشت، میانگین را بدون احتساب مقادیر null برگردانید.

نکته: count نشان دهنده ی تعداد دانشجویان فعلی است.

۲. `findTopStudent(Student[] students, int count)`

◦ نام دانشجویی با بالاترین نمره را پیدا کرده و آن را برمی گرداند.

◦ اگر count برابر صفر باشد، `EmptyStudentListException` پرتاب کند.

۳. `findStudentByName(Student[] students, int count, String name)`

◦ دانشجو را بر اساس نام پیدا کرده و او را برمی گرداند.

◦ اگر دانشجو پیدا نشد، استثنای سفارشی `StudentNotFoundException` بسازید و پرتاب

کنید

۴. `addStudent(Student[] students, int currentCount, Student newStudent)`

◦ دانشجو را به آرایه اضافه می کند.

◦ قبل از اضافه کردن، بررسی کند دانشجویی با نام تکراری وجود نداشته باشد.

◦ اگر تکراری بود، `DuplicateStudentException` پرتاب کند

◦ اگر آرایه پر بود، `ArrayFullException` پرتاب کند.

◦ این متد هیچ مقداری برنمی گرداند.

مثال

ورودی نمونه ۱

```

1 public class Main {
2     public static void main(String[] args) {
3         Student[] students = new Student[5];
4         ScoreProcessor processor = new ScoreProcessor();
5
6         try {
7             Student s1 = new Student("Ali", 18.5);
8             Student s2 = new Student("Sara", 19.0);
9             Student s3 = new Student("Reza", 17.0);
10
11             processor.addStudent(students, 0, s1);
12             processor.addStudent(students, 1, s2);
13             processor.addStudent(students, 2, s3);
14
15             double avg = processor.calculateAverage(students, 3);

```

```
        System.out.println("Average: " + avg);

        String top = processor.findTopStudent(students, 3);
        System.out.println("Top Student: " + top);

        Student found = processor.findStudentByName(students, 3, "Sara");
        System.out.println("Found: " + found.getName());

        Student s4 = new Student("Ali", 20.0);
        processor.addStudent(students, 3, s4);

    } catch (DuplicateStudentException e) {
        System.out.println("Exception caught: " + e.getMessage());
    } catch (Exception e) {
        System.out.println("Other error: " + e.getMessage());
    }
}
}
```

خروجی نمونه ۱

```
Average: 18.166666666666668
Top Student: Sara
Found: Sara
Exception caught: Student with name 'Ali' already exists
```

آنچه که باید آپلود کنید

شما باید یک فایل zip آپلود کنید که شامل کلاس های زیر باشد:

```
NegativeScoreException.java
InvalidScoreException.java
EmptyStudentListException.java
DuplicateStudentException.java
InvalidNameException.java
StudentNotFoundException.java
ArrayFullException.java
Student.java
ScoreProcessor.java
```


سیستم تراکنش‌های بانکی

- سطح: متوسط

- طراح: رامتین بدری

می‌توانید کلاس‌های اولیه پروژه را از [این لینک](#) دانلود کنید.

یک بانک نیاز به سیستمی دارد تا عملیات انتقال وجه بین حساب‌ها را مدیریت کند. در این سیستم، حفظ **سازگاری اطلاعات (Consistency)** و **اتمیک بودن تراکنش‌ها (Atomicity)** اهمیت بسیار زیادی دارد.

هر تراکنش باید به صورت کامل انجام شود. یعنی:

- یا تمام مراحل انتقال پول با موفقیت انجام شوند، یا در صورت بروز خطا، تمام تغییرات به حالت قبل بازگردند.

شما باید با استفاده از مفاهیم **Exception Handling**، طراحی Exception اختصاصی، و مدیریت rollback، این سیستم را پیاده‌سازی کنید. به بازگرداندن اطلاعات به وضعیت قبل از شروع تراکنش، **rollback** گفته می‌شود.

فیلد کمکی برای کلاس‌ها مجاز است.

کلاس Account

هر حساب بانکی شامل اطلاعات زیر است:

۱. id از نوع String (شناسه حساب)

۲. ownerName از نوع String (نام صاحب حساب)

۳. balance از نوع long (موجودی حساب)

۴. dailyTransferredAmount از نوع long (مجموع انتقال‌های امروز)

۵. active از نوع boolean (وضعیت فعال بودن حساب)

سازنده:

```
1 | public Account(String id, String ownerName, long balance) {
```

```
// TODO
}
```

متدهای موردنیاز:

```
1 | public void deposit(long amount)
```

مبلغ را به حساب اضافه می‌کند. برای شبیه‌سازی خطاهای غیرمنتظره سیستمی، اگر id حساب برابر "FAIL" باشد، متدهای deposit و withdraw باید هنگام اجرا یک RuntimeException پرتاب کنند و موجودی حساب بدون تغییر باقی بماند.

```
1 | public void withdraw(long amount)
```

مبلغ را از حساب کم می‌کند. برای شبیه‌سازی خطاهای غیرمنتظره سیستمی، اگر id حساب برابر "FAIL" باشد، متدهای deposit و withdraw باید هنگام اجرا یک RuntimeException پرتاب کنند و موجودی حساب بدون تغییر باقی بماند.

```
1 | public String getAccountInfo()
```

رشته‌ای با فرمت زیر برمی‌گرداند:

```
<ID> | <Owner> | Balance: <balance> | Active: <true/false>
```

کلاس Transaction

هر تراکنش شامل اطلاعات زیر است:

۱. sourceAccount از نوع Account (حساب مبدأ)
۲. destinationAccount از نوع Account (حساب مقصد)
۳. amount از نوع long (مقدار تراکنش)
۴. successful از نوع boolean (وضعیت موفق بودن تراکنش)

سازنده:

```
1 public Transaction(Account sourceAccount, Account destinationAccount, long am  
2     // TODO  
3 }
```

کلاس انتزاعی BankException

تمام Exception های پروژه باید از این کلاس ارث‌بری کنند. سازنده:

```
1 public BankException(String message) {  
2     super(message);  
3 }
```

اکسپشن های پروژه

```
1 DailyLimitExceededException  
2 InvalidAmountException  
3 InsufficientFundsException  
4 InactiveAccountException  
5 NullAccountException  
6 SameAccountException  
7 TransactionRollbackException
```

کلاس TransactionManager

این کلاس مسئول مدیریت تراکنش‌ها است.

ویژگی‌ها:

۱. transactions از نوع آرایه ای از Transaction ها

۲. DAILY_TRANSFER_LIMIT ثابت برابر 100000

نکته: تضمین می‌شود تعداد تراکنش‌ها بیشتر از 100 نخواهد شد.

سازنده:

```
1 public TransactionManager() {
```

```
// TODO  
}
```

قوانین انتقال

تراکنش فقط در شرایط زیر معتبر است:

۱. حساب مبدأ و مقصد نباید null باشند.
۲. مبلغ انتقال باید مثبت باشد.
۳. حساب مبدأ و مقصد نباید یکسان باشند.
۴. هر دو حساب باید active باشند.
۵. حساب مبدأ باید موجودی کافی داشته باشد.
۶. مجموع انتقال‌های روزانه حساب مبدأ نباید از DAILY_TRANSFER_LIMIT بیشتر شود.

متدهای TransactionManager

```
1 public void transfer(Account from, Account to, long amount) throws BankExcept  
2     // TODO  
3 }
```

این متد وظیفه انتقال پول بین دو حساب را دارد. در این متد شما باید از رعایت شدن قوانین انتقال مطمئن شوید و در صورت نقض هر کدام، باید Exception متناظر پرتاب شود:

۱. NullAccountException اگر حساب مبدأ یا مقصد برابر null باشد.
۲. InvalidAmountException اگر مبلغ انتقال کوچکتر مساوی صفر باشد.
۳. SameAccountException اگر حساب مبدأ و مقصد یکسان باشند.
۴. InactiveAccountException اگر یکی از حساب‌ها active نباشد.
۵. InsufficientFundsException اگر موجودی حساب مبدأ کافی نباشد.
۶. DailyLimitExceededException اگر مجموع انتقال‌های روزانه حساب مبدأ از DAILY_TRANSFER_LIMIT بیشتر شود.

اتمیک بودن تراکنش‌ها

این سیستم باید Atomic باشد.

یعنی اگر هنگام انجام عملیات انتقال، **بعد از تغییر وضعیت حساب‌ها** Exception رخ دهد:

- موجودی حساب‌ها باید به وضعیت قبل بازگردد.
- هیچ تغییری نباید باقی بماند.
- تراکنش باید به صورت ناموفق در لیست transactions ذخیره شود.
- و در نهایت TransactionRollbackException پرتاب شود.

```
1 public String[] getTransactionReports() {  
2     // TODO  
3 }
```

گزارش تمام تراکنش‌ها را برمی‌گرداند.

فرمت:

<FROM> -> <TO> | Amount: X | SUCCESS

یا:

<FROM> -> <TO> | Amount: X | FAILED

```
1 public long getTotalTransferredAmount() {  
2     // TODO  
3 }
```

مجموع تمام انتقال‌های موفق را برمی‌گرداند.

```
1 public int getFailedTransactionCount() {  
2     // TODO  
3 }
```

تعداد تراکنش‌های ناموفق را برمی‌گرداند.

```
1 public void deactivateAccount(Account account) {  
2     // TODO  
3 }
```

حساب را غیرفعال می‌کند.

مثال

پروژه را به گونه‌ای پیاده‌سازی کنید که با اجرای Main زیر:

```
1 public class Main {  
2     public static void main(String[] args) {  
3  
4         TransactionManager manager = new TransactionManager();  
5  
6         Account a1 = new Account("A100", "Ali", 50000);  
7         Account a2 = new Account("B200", "Sara", 20000);  
8  
9         try {  
10            manager.transfer(a1, a2, 10000);  
11        }  
12        catch (BankException e) {  
13            System.out.println(e.getMessage());  
14        }  
15  
16        System.out.println(a1.getAccountInfo());  
17        System.out.println(a2.getAccountInfo());  
18  
19        for(String report : manager.getTransactionReports()){  
20            System.out.println(report);  
21        }  
22    }  
23 }
```

خروجی به شکل زیر باشد:

```
A100 | Ali | Balance: 40000 | Active: true  
B200 | Sara | Balance: 30000 | Active: true  
A100 -> B200 | Amount: 10000 | SUCCESS
```

آنچه که باید آپلود کنید

شما باید یک فایل zip آپلود کنید که ساختاری مشابه زیر داشته باشد:

```
├─ Account.java  
├─ Transaction.java  
├─ TransactionManager.java  
├─  
└─ Exceptions/  
    ├─ BankException.java  
    ├─ DailyLimitExceededException.java  
    ├─ InvalidAmountException.java  
    ├─ InsufficientFundsException.java  
    ├─ InactiveAccountException.java  
    ├─ NullAccountException.java  
    ├─ SameAccountException.java  
    └─ TransactionRollbackException.java
```

Comic-Con

- سطح: متوسط
- طراح: آراین صیدی

در این سوال قصد داریم با توجه به مفاهیم پیشرفته‌ی جنریک‌ها (Generics) و مدیریت استثناءها (Exceptions) کلاس‌های یک سیستم مدیریت استیج برای رویداد Comic-Con را پیاده‌سازی کنیم:

ساختار فایل‌ها

```

1 | ComicCon.zip
2 | └─ ShowcaseItem.java
3 |   └─ VideoGame.java
4 |     └─ Movie.java
5 | └─ EventStage.java
6 | └─ StageFullException.java
7 | └─ IdenticalStageMergeException.java
8 | └─ NoAvailableShowcase.java
9 | └─ AlreadyAnnounced.java

```

برای دانلود فایل اولیه پروژه کلیک کنید 

جزئیات پروژه

کلاس ShowcaseItem (از نوع abstract):

این کلاس نماینده هر برنامه‌ای روی استیج است و دارای ویژگی‌های زیر می‌باشد:

۱. **فیلد title**: از نوع String برای نگهداری عنوان برنامه.
۲. **فیلد announced**: از نوع boolean برای بررسی وضعیت پخش برنامه.
۳. **متد void Announcement() throws AlreadyAnnounced**: در صورتی که برنامه قبلاً پخش شده باشد، خطای AlreadyAnnounced پرتاب می‌کند.

کلاس های Movie , VideoGame :

این دو کلاس فرزندان ShowcaseItem هستند؛ با توجه به مشخصات زیر آن‌ها را پیاده‌سازی کنید:

کلاس VideoGame 🎮: علاوه بر فیلد عنوان، دارای یک فیلد ثابت به نام studio (از نوع final String) است که از طریق سازنده مقداردهی می‌شود.

کلاس Movie 🎬: علاوه بر فیلد عنوان، دارای یک فیلد به نام durationMins (از نوع int) برای نگهداری مدت زمان فیلم است.

کلاس EventStage :

این کلاس یک کلاس **جنریک** است و مسئولیت مدیریت کنداکتور (لیست پخش) یک استیج را بر عهده دارد.

ویژگی‌ها و سازنده:

۱. **فیلد lineup:** آرایه‌ای جنریک برای نگهداری برنامه‌ها.
۲. **فیلد count:** تعداد آیتم‌های فعلی موجود در استیج.
۳. **سازنده:** در متد سازنده، آرایه lineup را با **ظرفیت اولیه ثابت ۲۰** ایجاد کنید.

متدها:

1. متد addContent :

```
1 | public boolean addContent(T item) throws StageFullException
```

برنامه جدید را به کنداکتور اضافه می‌کند. اگر ظرفیت آرایه پر شده باشد خطای StageFullException و اگر آیتم ورودی نامعتبر (null) باشد استثنای NullPointerException پرتاب می‌کند؛ در غیر این صورت مقدار true برمی‌گرداند.

2. متد startShow :

```
1 | public int startShow()
```

مراسم آغاز می‌شود! این متد روی تمام برنامه‌های موجود در استیج را با استفاده از متد

`Announcement()` معرفی میکند.

- اگر استیج کاملاً خالی باشد، خطای `NoAvailableShowcase` پرتاب شود.
- در یک پخش زنده، اگر یک برنامه قبلاً لو رفته باشد (خطای `AlreadyAnnounced` پرتاب شود)، کل مراسم نباید متوقف شود و باید ادامه یابد.
- در نهایت، این متد باید **تعداد برنامه‌هایی که با موفقیت پخش شدند** را `return` کند.

3. متد `getPresentation`:

```
1 | public T getPresentation(int index)
```

آیتم روی ایندکس مشخص شده را برمی‌گرداند (بررسی اعتبار ایندکس باید بر اساس تعداد آیتم‌های واقعی انجام شود و در صورت نامعتبر بودن، خطای `IndexOutOfBoundsException` پرتاب شود).

4. متد `mergeSchedule`:

```
1 | public void mergeSchedule(EventStage<? extends /*TODO*/> ShowcaseItems)
```

برنامه‌های یک استیج دیگر را دریافت کرده و به استیج فعلی اضافه می‌کند. **جلوگیری از ادغام با خود:** اگر استیج داده شده با استیج فعلی یکی بود، باید استثنای `IdenticalStageMergeException` پرتاب شود.

- متد ادغام نباید هیچ خطایی پرتاب کند. شما باید برنامه‌های مهمان را با استفاده از متد `addContent` اضافه کنید. اگر در حین اضافه کردن متوجه شدید ظرفیت پر شده است و متد `addContent` خطای `StageFullException` را پرتاب کرد، باید خطای داده شده را مدیریت کنید، یک ارایه جدید با ظرفیت مناسب بسازید و برنامه‌های مهمان را اضافه کنید.

5. متد `replaceItem`:

```
1 | public <E extends /*TODO*/> void replaceItem(int index, E newItem)
```

یک متد جنریک مستقل است که برنامه جدید را در ایندکس مشخص شده جایگزین می‌کند. (در صورت نامعتبر بودن ایندکس، `IndexOutOfBoundsException` پیش‌فرض جاوا پرتاب شود).

منظور از `/*TODO*/` در ورودی متد، استفاده از Upper Bounds است تا نوع ورودی محدود شود.

کلاس های استثنا (Exceptions):

- StageFullException (از نوع Checked - ارث‌بری از Exception)
- AlreadyAnnounced (از نوع Unchecked - ارث‌بری از RuntimeException)
- IdenticalStageMergeException (از نوع Unchecked - ارث‌بری از RuntimeException)
- NoAvailableShowcase (از نوع Unchecked - ارث‌بری از RuntimeException)

MiniUnit

- سطح: سخت

- طراح: محمد امین مولائی

به دلیل اختلال در شبکه اینترنت بین‌الملل، دسترسی دانشجویان دانشکده به مخازن Maven و Gradle قطع شده است و امکان دانلود کتابخانه‌های تست واحد مثل **JUnit** وجود ندارد. از آنجایی که پروژه درس «برنامه‌نویسی پیشرفته» بدون ابزار تست پیش نمی‌رود، تیم آموزشی تصمیم گرفته است که شما را مأمور پیاده‌سازی یک نسخه بومی و ساده‌سازی شده از فریم‌ورک تست به نام **MiniUnit** کند. شما باید با استفاده از رفلکشن برنامه‌ای بنویسید که بتواند کلاس‌های تست را تحلیل کند، متدهای تست را پیدا و آن‌ها را اجرا کند.

می‌توانید کلاس‌های اولیه پروژه را از [این لینک](#) دانلود کنید.

در این پروژه، بیشتر کلاس‌ها از قبل پیاده‌سازی شده‌اند و شما فقط باید کلاس `TestRunner` را کامل کنید.



قوانین شناسایی و اجرای تست‌ها

1. متدهای تست:

هر متدی که شرایط زیر را داشته باشد یک متد تست محسوب می‌شود.

- نام آن با عبارت `test` شروع شود و اولین حرف بعد از آن بزرگ باشد (مثلاً `testAddition`).
- هیچ پارامتر ورودی نداشته باشد.
- نوع خروجی آن `void` باشد.
- نباید `static` باشد.

نکته: این متدها می‌توانند `public` یا `private` باشند. برنامه شما باید بتواند هر دو را اجرا کند.

2. مدیریت چرخه حیات (Lifecycle):

فریم‌ورک شما باید از یک متد ویژه پشتیبانی کند:

- `beforeEach` : اگر متدی با نام `beforeEach` بدون پارامتر و با نوع بازگشتی `void` وجود داشته باشد، باید دقیقاً قبل از اجرای هر متد تست اجرا شود.

نکته 1: این متد نیز ممکن است `public` یا `private` باشد و نباید `static` باشد.

نکته 2: اگر `beforeEach` یا متد تست هر نوع `Throwable` (شامل `Exception` ها و `Error` ها) پرتاب کنند، آن تست شکست خورده محسوب می‌شود. در صورتی که `beforeEach` شکست بخورد، اجرای متد تست متوقف شده و متد تست اجرا نخواهد شد.

نکته 3: تضمین می‌شود در هر کلاس حداکثر یک `beforeEach` معتبر وجود داشته باشد.

3. جداسازی تست‌ها:

برای جلوگیری از تداخل تست‌ها، **هر تست باید روی یک شیء مجزا اجرا شود**. یعنی برای هر متد تست، باید یک نمونه جدید از کلاس تست بسازید، سپس `beforeEach` را اجرا کنید و در نهایت خود تست را اجرا کنید.

4. ترتیب اجرا:

از آنجایی که **رفلکشن ترتیب متدها را تضمین نمی‌کند**، برای یکسان بودن خروجی، باید متدهای تست

شناسایی شده را بر اساس **نام متد** به ترتیب الفبا مرتب کنید. (متد مرتب‌سازی در کد اولیه موجود است).

کلاس Assert

کلاس Assert شامل متدهای استاتیکی است که در صورت برقرار نبودن شرط تست، یک `AssertionError` پرتاب می‌کنند.

برای نمونه یکی از این متدها را بررسی می‌کنیم. بقیه متدها کارکرد مشابهی دارند.

`AssertEquals` : برای سادگی، نوع آرگومان‌ها فقط می‌تواند یکی از انواع `int`، `double` یا `String` باشد. اگر دو ورودی با هم برابر نباشند، یک `AssertionError` پرتاب می‌شود.

کلاس TestResult

وظیفه نگهداری نتایج (نام تست‌های پاس‌شده، نام و پیام خطای تست‌های شکست‌خورده) و تولید گزارش نهایی را بر عهده دارد. این کلاس دارای ویژگی‌های زیر است.

۱. `passedTests` آرایه‌ای از نوع `String` که نام متدهای پاس‌شده را نگه می‌دارد
۲. `failedTests` آرایه‌ای از نوع `String` که نام متدهای شکست‌خورده را نگه می‌دارد
۳. `failureMessages` آرایه‌ای از نوع `String` که پیام‌های تست‌های شکست‌خورده را نگه می‌دارد
۴. `passedCount` از نوع `int` برای تعداد متدهای پاس‌شده
۵. `failedCount` از نوع `int` برای تعداد متدهای شکست‌خورده

مهم‌ترین متدهای این کلاس به شرح زیر است.

```
1 | public void addPass(String testName)
```

اگر تست پاس شود، این متد فراخوانی می‌شود و نام تست به تست‌های پاس‌شده اضافه می‌شود.

```
1 | public void addFailure(String testName, String message)
```

اگر تست شکست بخورد، این متد فراخوانی می‌شود. این متد نام تست و پیام مربوط به آن را می‌گیرد و به

تست‌های شکست‌خورده اضافه می‌کند.

```
1 | public String createReport()
```

این متد گزارش نهایی تست را به صورت String برمی‌گرداند.

کلاس TestRunner

کلاس اصلی فریم‌ورک که وظیفه اصلی بر عهده آن است و شما باید آن را کامل کنید. در این کلاس باید با استفاده از رفلکشن تست‌ها را اجرا کنید.

```
1 | public static TestResult runClass(String className) {  
2 |     //TODO  
3 | }
```

این متد تنها متد public این کلاس است و به صورت static تعریف شده است. شما باید متد را با کمک متدهای کمکی پیاده‌سازی کنید.

این متد نام کامل کلاس را می‌گیرد، آن را بارگذاری (Load) کرده و چرخه اجرای تست‌ها را مدیریت می‌کند. تضمین می‌شود اگر کلاس تست با این نام وجود داشته باشد، سازنده بدون آرگومان دارد. اگر کلاسی با این نام یافت نشد یک RuntimeException با پیغام زیر پرتاب می‌شود.

Test class not found: <class_name>

```
1 | private static boolean isTestMethod(Method method) {  
2 |     //TODO  
3 | }
```

یک متد به عنوان ورودی می‌گیرد و با توجه به قوانین گفته شده چک می‌کند که یک متد تست است یا نه.

```
1 | private static Method[] findTestMethods(Class<?> testClass) {  
2 |     //TODO  
3 | }
```

تمامی متدهایی که شرایط تست را دارند استخراج کرده، با استفاده از متد `sortMethodsByName` مرتب می‌کند و برمی‌گرداند. دقت کنید که فقط متدهای تعریف‌شده در همان کلاس تست بررسی می‌شوند.

```
1 | private static Method findBeforeEach(Class<?> testClass) {  
2 |     //TODO  
3 | }
```

با گرفتن یک کلاس، متد `beforeEach` را برمی‌گرداند. اگر کلاس این متد را نداشت `null` برگردانده می‌شود.

```
1 | private static void callMethod(Object object, Method method) throws Throwable  
2 |     //TODO  
3 | }
```

اجرای متد با استفاده از رفلکشن را انجام می‌دهد. دقت کنید که برای اجرای متدهای `private` باید دسترسی لازم را ایجاد کنید.

```
1 | private static Object createTestObject(Class<?> testClass) throws Throwable {  
2 |     //TODO  
3 | }
```

طبق قوانین گفته شده برای جلوگیری از تداخل تست‌ها با هم، هر تست باید روی یک شیء مجزا از کلاس تست انجام شود. با کمک این متد می‌توان از روی کلاس تست شیء جدید ایجاد کرد.

```
1 | private static void runOneTest(  
2 |     Class<?> testClass,  
3 |     Method testMethod,  
4 |     Method beforeEachMethod,  
5 |     TestResult result) {  
6 |  
7 |     //TODO  
8 | }
```

مسئول ساخت شیء، فراخوانی `beforeEach` و متد تست، و ثبت نتیجه در `result` است.

```
1 | private static String createFailureMessage(Throwable problem)
```

این متد پیام مربوط به شکست خوردن تست را برمی‌گرداند. این متد پیاده‌سازی شده و نیاز به تغییر آن ندارید.

```
1 | private static void sortMethodsByName(Method[] methods)
```

آرایه‌ای از متدها به عنوان ورودی می‌گیرد و آن‌ها را بر اساس نام به ترتیب حروف الفبا مرتب می‌کند. این متد پیاده‌سازی شده و نیاز به تغییر آن ندارید.

این متد با الگوریتم مرتب‌سازی حبابی (Bubble Sort) پیاده‌سازی شده است. برای مطالعه بیشتر در مورد این الگوریتم مرتب‌سازی می‌توانید [این فایل](#) را مطالعه کنید.

برای درک بهتر سوال، تست موجود در پروژه اولیه را بررسی و اجرا کنید.

برای کلاس Calculator کلاس تست CalculatorTest مطابق با استانداردهای MiniUnit نوشته شده است. با اجرای متد main کلاس MiniUnit انتظار داریم خروجی زیر را ببینیم:

```
=====
Test report for: CalculatorTest
=====
[PASS] testAddTwoPositiveNumbers
[PASS] testAverageOfTwoNumbers
[PASS] testMemoryStoresValue
[PASS] testSubtractTwoNumbers
[FAIL] testCrash
      Test crashed with message: Cannot invoke "Calculator.add(int, int)" because
[FAIL] testFail
      forced to fail.
[FAIL] testPrivateMultiplyMethod
      Expected 20 but got 21.
[FAIL] testTrueFalse
      Expected true but got false.
-----
Total:  8
Passed: 4
Failed: 4
=====
```

آنچه که باید آپلود کنید

شما فقط باید کلاس تکمیل شده `TestRunner.java` را آپلود کنید.